

OBJECT FOLLOWING ROBOT USING ARDUINO UNO

Mini Project Report Submitted in partial fulfilment of the requirements for the degree
of Bachelor of Technology from Maulana Abul Kalam Azad University of
Technology, West Bengal (formerly known as West Bengal University of Technology)

By

Aunupam Dutta (Roll No. 34900323003)

Biraj Sarkar (Roll No. 34900323013)

Debanjan Chakraborty (Roll No. 34900323015)

Sayan Sarkar (Roll No. 34900323048)

Soumyajit Pandit (Roll No. 34900323052)

Under the Guidance of

Project Supervisor: Dr. Gautam Das

Department of Electronics and Communication Engineering

Cooch Behar Government Engineering College

Cooch Behar, West Bengal

2026

Acknowledgement

First and foremost, we would like to express our deep sense of gratitude to our project supervisor, Dr. Gautam Das, and the Department of Electronics and Communication Engineering at Cooch Behar Government Engineering College for providing us with the opportunity to work on this interesting mini project. His constant guidance, technical feedback, and continuous support have been instrumental in the successful completion of this design and prototype realization.

We are also very thankful to the laboratory instructors who provided the necessary electronic components, testing equipment, and workspace to construct and test the robotic vehicle. Finally, we would like to thank our families, friends, and peers for their constant encouragement and support throughout this work, which has given us valuable experience in hardware-software integration, sensor interfacing, and wireless control protocols.

Project Members

Roll No.	Name of the Member	Signature of the Member
34900323003	Aunupam Dutta	
34900323013	Biraj Sarkar	
34900323015	Debanjan Chakraborty	
34900323048	Sayan Sarkar	
34900323052	Soumyajit Pandit	

Publications and Acronyms

No academic research papers, patents, or conference presentations have been published from the research work and design findings presented in this mini project report so far. The system has been designed and implemented primarily as an educational prototype to fulfill the academic requirements of the Bachelor of Technology degree in Electronics and Communication Engineering.

The following table lists the principal abbreviations, symbols, acronyms, and technical terms used in this mini project report, along with their definitions.

Acronym / Symbol	Description / Meaning
ECE	Electronics and Communication Engineering
UNO	Arduino Development Board based on the ATmega328P microcontroller
PWM	Pulse Width Modulation (used to control motor speed)
IR	Infrared (electromagnetic radiation used in proximity detection)
SR04	Model number of the ultrasonic distance sensor used in this design
GND	Electrical Common Ground
VCC	Voltage at the Common Collector (Positive Power Supply)
GPIO	General Purpose Input / Output pins
ADC	Analog-to-Digital Converter

Table 1: List of Acronyms and Symbols

Contents

<i>Acknowledgement</i>	<i>i</i>
<i>Publications and Acronyms</i>	<i>ii</i>
<i>Lists of Figures and Tables</i>	<i>iv</i>
1 Introduction	1
1.1 Project Overview	1
1.2 Review of Previous Works	1
1.3 Objective and Motivation	2
1.4 Scope of Present Work	2
2 Hardware Architecture and Component Details	3
2.1 Microcontroller	3
2.2 Sensors	3
2.3 Actuators and Gearing	4
2.4 Motor Driver Module	5
2.5 Power Source and Battery Configuration	5
3 System Design and Circuit Integration	6
3.1 System Architecture	6
3.2 Arduino UNO Pin Configurations	6
3.3 Schematic Connections and Interfaces	7
3.4 Kinematics of Differential Drive	7
3.5 Power Scheme and Ground Isolation	8
4 Software Implementation and Control Algorithms	9
4.1 Development Environment	9
4.2 Control Algorithm Logic	9
4.3 Control Firmware Source Code	9
5 Experimental Results, Conclusions and Future Scope	11
5.1 System Calibration and Testing	11
5.2 Performance Evaluation	11
5.3 Conclusions	12
5.4 Future Scope	12
References	13

Lists of Figures and Tables

List of Figures

2.1	Arduino UNO Development Board	3
2.2	HC-SR04 Ultrasonic Sensor	4
2.3	Infrared Proximity Sensor Module	4
2.4	Brushed BO Geared Motor	4
2.5	L298N H-Bridge Motor Driver Module	5
3.1	Object Following Robot System Block Diagram	6
5.1	Assembled Object Following Robot	11
5.2	Internal Chassis Wiring and Layout	11
5.3	Ranging and Sensor Test Setup	12
5.4	Battery Mount	12

List of Tables

1	List of Acronyms and Symbols	ii
1.1	Project System Specifications	2
3.1	Arduino UNO Physical Pin Connections	7
5.1	Sensor Calibration and Command Mapping	11

Abstract

This mini project presents the design and construction of an autonomous Object Following Robot using the Arduino UNO microcontroller. The robot is designed to detect and follow a target object or person in real time while keeping a safe distance. The sensing unit consists of an HC-SR04 ultrasonic distance sensor mounted in a fixed position on the front of the robot chassis, which measures the distance to the target. To help with distance tracking, two infrared (IR) proximity sensors are mounted on the left and right sides of the chassis. These IR sensors detect if the target moves sideways, allowing the robot to turn and follow the target dynamically.

The robot is driven by four battery-operated (BO) geared motors controlled by an L298N dual H-bridge motor driver. The Arduino UNO reads the sensor signals, calculates the distance and direction of the target, and controls the motor speed using Pulse Width Modulation (PWM). If the target goes out of range, the robot stops and waits for it to reappear. The system is powered by a 7.4V battery pack made of two 18650 Lithium-Ion cells with a shared common ground. We tested the assembled prototype extensively. The results show that the robot follows targets reliably between 10 cm and 35 cm, backs up to recover a safe distance if the target gets closer than 10 cm, and turns smoothly to track lateral movements.

Chapter 1 : Introduction

1.1 Project Overview

Autonomous mobile robots have emerged as a cornerstone of modern industrial automation, logistics, and assistive technologies. Among these, human-following and object-following capabilities represent an essential domain of human-machine interaction (HMI). An object-following robot, by contrast, tracks and follows a specific target autonomously, enabling hands-free operation and natural collaboration between humans and machines. Such systems are highly applicable in smart shopping carts, automated warehouse transport, medical aid devices, and personal assistant platforms [1].

This project focuses on the development of an intelligent, low-cost Object Following Robot built on the Arduino UNO microcontroller platform. The robotic platform integrates an HC-SR04 ultrasonic sensor for forward distance detection and dual IR sensors for lateral target tracking. By processing distance inputs and lateral sensor states, the Arduino UNO regulates the differential drive system to follow a target while maintaining a safety buffer. The system combines ultrasonic acoustic ranging with infrared proximity detection, showing how multi-sensor configurations can improve tracking reliability on a simple mobile platform.

1.2 Review of Previous Works

In early mobile robotics, target following relied on complex wireless beacons, such as radio frequency (RF) transmitters, infrared emitters, or magnetic induction loops. The emergence of vision-based human following represented a significant advancement, utilizing digital cameras and computer vision algorithms (such as color histogram matching, optical flow, and deep learning) to detect human silhouettes. However, vision-based architectures demand high computational resources, typically requiring an onboard Raspberry Pi or single-board computer, which increases cost, power consumption, and processing latency [2].

To address cost and power constraints, researchers have explored active sensor configurations. Acoustic ranging using ultrasonic sensors provides a reliable method for measuring distance in real time. Unlike optical sensors, ultrasonic sensors are unaffected by ambient light variations, making them suitable for indoor environments. Proximity tracking using infrared sensors offers a simple method for detecting lateral deviation. Recent literature demonstrates that combining ultrasonic distance measurement with dual infrared sensors creates an effective sensory envelope for short-range

following applications [3]. This project builds upon these concepts by integrating a fixed ultrasonic sensor with side-mounted infrared sensors to establish a robust following algorithm.

1.3 Objective and Motivation

The primary objective of this mini project is to design, construct, and evaluate an autonomous, low-cost Object Following Robot that maintains a fixed distance from a leading target. The system must process inputs from distance and proximity sensors and execute real-time steering corrections. The key motivation is to design a simple, robust human-following system using low-power hardware. By using an Arduino UNO and avoiding expensive vision processors, we show that effective tracking can be achieved through coordinated sensor control and differential drive algorithms. This research is motivated by applications in assistive logistics, such as smart hospital carts that follow medical staff, and hands-free warehouse systems.

1.4 Scope of Present Work

The present work covers the complete design, assembly, and testing of the object-following robotic vehicle. The scope includes:

1. Interfacing the HC-SR04 ultrasonic sensor with the Arduino UNO to calculate target distance.
2. Integrating dual IR obstacle sensors to detect lateral deviation and trigger turning maneuvers.
3. Designing a differential drive steering algorithm using an L298N H-Bridge motor driver and four BO motors.
4. Calibrating threshold distances to define safe movement states.
5. Developing a common grounding scheme to stabilize microcontroller logic rails.

Feature	Arduino UNO Following Robot Specifications
Microcontroller	Arduino UNO Board (ATmega328P, 8-bit RISC, 16 MHz)
Primary Sensor	HC-SR04 Ultrasonic Distance Sensor (2cm – 400cm range)
Lateral Sensors	Dual IR Proximity Sensors (Adjustable detection threshold)
Motor Driver	L298N Dual H-Bridge Driver (2A peak per channel)
Actuators	Four 150 RPM Dual Shaft BO Geared Motors
Power Source	Dual 18650 Li-ion Cells (7.4V, 2200 mAh)
Chassis Design	4WD Differential Drive Chassis
Typical Latency	10 – 15 ms

Table 1.1: Project System Specifications

Chapter 2 : Hardware Architecture and Component Details

2.1 Microcontroller

The core processing unit of the robot is the Arduino UNO development board, based on the ATmega328P microcontroller [1]. The ATmega328P is an 8-bit RISC-based processor operating at a clock frequency of 16 MHz. It contains 32 KB of Flash memory, 2 KB of SRAM, and 1 KB of EEPROM. The board features 14 digital input/output pins (of which 6 support hardware PWM outputs) and 6 analog inputs. The Arduino UNO processes inputs from the ultrasonic and IR sensors, calculates target distance and direction, and generates PWM signals to control motor speed. Figure 2.1 shows the development board.



Figure 2.1: Arduino UNO Development Board

2.2 Sensors

The robot's tracking system uses an ultrasonic distance sensor for forward tracking and dual infrared sensors for lateral target detection.

1. **HC-SR04 Ultrasonic Sensor:** The HC-SR04 measures target distance by emitting ultrasonic pulses [2]. It transmits an 8-cycle burst of ultrasound at 40 kHz, which propagates through the air. When the wave hits an object, it reflects back as an echo. The sensor's receiver detects this echo and outputs a high pulse on the Echo pin. The duration of this pulse corresponds to the wave's round-trip travel time. The distance d is calculated using the speed of sound (340 m/s):

$$d = \frac{t \times 0.034}{2} \quad (2.1)$$

where t is the echo duration in microseconds. The sensor operates at 5V, measuring distances from 2 cm to 400 cm with an accuracy of 3 mm. Figure 2.2 shows the ultrasonic sensor.



Figure 2.2: HC-SR04 Ultrasonic Sensor

2. **IR Proximity Sensors:** Two IR sensors are mounted on the left and right sides of the chassis. Each sensor has an infrared LED transmitter and a photodiode receiver. The transmitter emits infrared light. If an object is nearby, the light reflects back to the photodiode, decreasing its resistance. An LM393 comparator chip processes this signal and outputs a digital LOW when an obstacle is detected. A potentiometer on the board adjusts the detection range (typically 2 cm to 10 cm). Figure 2.3 shows the IR proximity sensor module.



Figure 2.3: Infrared Proximity Sensor Module

2.3 Actuators and Gearing

The mechanical movement of the robot is driven by battery-operated geared motors. The robot uses four BO geared motors. These motors operate between 3V and 9V. They feature a 1:48 gear reduction ratio, providing a stable speed of 150 RPM at 6V with high torque. This torque is necessary to steer the 4-wheel differential drive chassis. Figure 2.4 shows the brushed BO geared motor.



Figure 2.4: Brushed BO Geared Motor

2.4 Motor Driver Module

The high current demands of the motors require a dedicated driver. The L298N is a dual H-bridge motor driver that controls DC motors [4]. It contains two H-bridge circuits using high-power transistors to regulate motor direction and speed. The driver handles motor supply voltages up to 46V and supplies up to 2A per channel. Speed is regulated by applying PWM signals to the Enable pins (ENA, ENB), while direction is controlled via input pins (IN1–IN4). The four BO motors are wired in parallel pairs (left motors and right motors) to establish a 4-wheel differential drive system. Figure 2.5 shows the L298N H-Bridge motor driver module.

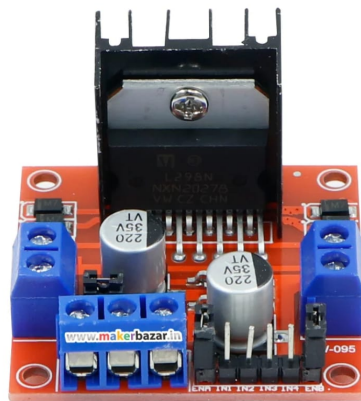


Figure 2.5: L298N H-Bridge Motor Driver Module

2.5 Power Source and Battery Configuration

The system is powered by two 18650 Lithium-Ion cells connected in series, providing a 7.4V battery pack. This power supply connects directly to the motor driver's power rail, while a regulated 5V line powers the logic circuitry of the Arduino UNO and sensory modules, ensuring stable operating conditions.

Chapter 3 : System Design and Circuit Integration

3.1 System Architecture

The system architecture of the Object Following Robot integrates the Arduino UNO with sensors and motor drivers. The Arduino UNO reads distance data from the HC-SR04 ultrasonic sensor and lateral tracking states from the dual IR sensors. It processes this data to compute control signals, steering the robot via the L298N driver. Figure 3.1 shows the system architecture block diagram.

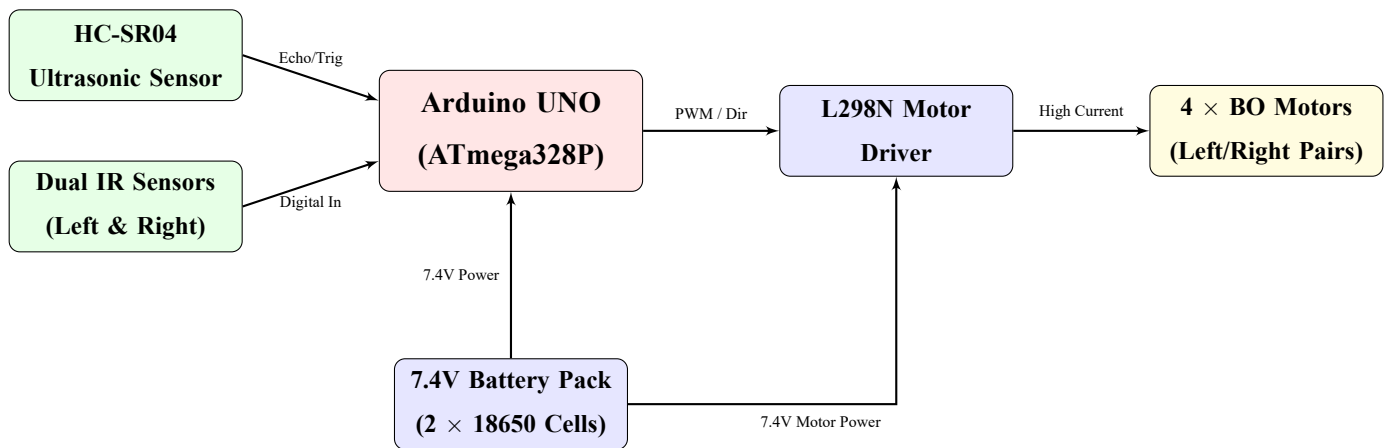


Figure 3.1: Object Following Robot System Block Diagram

3.2 Arduino UNO Pin Configurations

The pins on the Arduino UNO are mapped to organize connections. Table 3.1 lists the physical pin connections.

Arduino Pin	Connected Component	Signal Function
Digital D2	Right IR Sensor OUT	Digital input for right flank detection
Digital D3	Left IR Sensor OUT	Digital input for left flank detection
Digital D4	L298N IN1 Pin	Direction logic 1 for Left motor pair
Digital D5	L298N IN2 Pin	Direction logic 2 for Left motor pair
Digital D6	L298N IN3 Pin	Direction logic 1 for Right motor pair
Digital D7	L298N IN4 Pin	Direction logic 2 for Right motor pair
Digital D8	HC-SR04 TRIG Pin	Ultrasonic trigger pulse output (10 μ s)
Digital D10	L298N ENA Pin	PWM Speed control for Left motor pair
Digital D11	L298N ENB Pin	PWM Speed control for Right motor pair
Digital D13	HC-SR04 ECHO Pin	Ultrasonic echo pulse input (pulse duration)
5V Output Pin	Sensors VCC	Regulated 5V supply line
GND Output Pin	Common Ground GND	Shared ground reference for all modules

Table 3.1: Arduino UNO Physical Pin Connections

3.3 Schematic Connections and Interfaces

The system connections are split into the sensor input interface and the high-current motor control interface.

1. **Sensor Interfaces:** The trigger pin of the HC-SR04 is connected to D8, and the echo pin is connected to D13. The Arduino sends a trigger pulse to TRIG, causing the sensor to transmit acoustic waves. It then measures the echo duration on ECHO. The Left and Right IR sensor output pins are connected to D3 and D2 respectively. These pins pull to ground (LOW) when an obstacle is detected.
2. **Motor Driver Interface:** The L298N driver requires six control lines. ENA (D10) and ENB (D11) regulate motor speed via PWM. IN1/IN2 (D4/D5) control the direction of the left wheels, while IN3/IN4 (D6/D7) control the direction of the right wheels.

3.4 Kinematics of Differential Drive

The robot uses a 4-wheel differential drive steering system. Turning is achieved by varying the rotational speed and direction of the left and right wheel pairs. The robot's linear velocity v and angular velocity ω are defined by the speeds of the left and right wheels:

$$v = \frac{v_R + v_L}{2} \quad (3.1)$$

$$\omega = \frac{v_R - v_L}{W} \quad (3.2)$$

where v_L is the linear velocity of the left wheels, v_R is the linear velocity of the right wheels, and W is the track width of the robot chassis.

- **Forward Motion:** Left and right wheel pairs rotate forward at equal speeds ($v_L = v_R > 0$).
- **Backward Motion:** Left and right wheel pairs rotate in reverse at equal speeds ($v_L = v_R < 0$).
- **Pivot Left Turn:** Left wheels rotate in reverse and right wheels rotate forward ($v_R = -v_L > 0$).
- **Pivot Right Turn:** Right wheels rotate in reverse and left wheels rotate forward ($v_L = -v_R > 0$).

3.5 Power Scheme and Ground Isolation

Connecting DC motors and microcontrollers to the same power supply can cause issues. When DC motors start up or change directions, they draw a lot of current. This can drop the battery voltage and reset the Arduino UNO. To prevent this, we split the power lines. The 7.4V battery pack connects directly to the L298N motor driver to power the motors. The driver's internal regulator steps down the voltage to 5V to power the Arduino UNO. We also connected the grounds of the battery, Arduino, and motor driver together to establish a shared ground reference.

When we first built the prototype, we included a 0.96-inch SSD1306 OLED display and an SG90 micro servo motor. However, when the servo motor turned, it drew too much current and caused the voltage on the 5V line to drop. This voltage drop caused the Arduino to reset repeatedly and made the sensor readings unstable. To solve this power issue and make the robot run reliably on a single battery pack, we removed the display and the servo motor from the final design.

Chapter 4 : Software Implementation and Control Algorithms

4.1 Development Environment

The robot's control firmware was written in C++ using the Arduino IDE 2.x framework. The code uses standard microcontroller libraries alongside specialized sensor libraries:

- **Arduino.h**: Serves as the core library defining standard GPIO macros and IO functions.
- **NewPing.h**: Provides an optimized interface for the HC-SR04 sensor, handling trigger-echo timing without blocking execution.

4.2 Control Algorithm Logic

The robot's behavior is governed by a state machine that processes data from the ultrasonic and IR sensors:

1. **Forward Following**: If the target object is detected between the safety limit ($D_{MIN} = 10$ cm) and follow limit ($D_{MAX} = 35$ cm), and both IR sensors are HIGH, the robot drives forward.
2. **Lateral Steering**: If the right IR sensor is LOW (target detected on right), the robot steers right. If the left IR sensor is LOW, it steers left.
3. **Safety Recovery (Backup & Turn)**: If the target gets too close (< 10 cm), the robot stops, reverses, turns right, and stops to recover a safe distance.
4. **Search Mode**: If no target is detected within range, the robot pivots right to scan the surrounding area for objects.

4.3 Control Firmware Source Code

Listing 4.1 shows the annotated control firmware for the Arduino UNO.

```
1  #include <NewPing.h>
2
3  #define TRIG_PIN 8
4  #define ECHO_PIN 13
5  #define MAX_DISTANCE 200
6
7  #define RIGHT_IR 2
8  #define LEFT_IR 3
9
10 #define ENA 10
11 #define ENB 11
12 #define IN1 4
13 #define IN2 5
14 #define IN3 6
15 #define IN4 7
16
```

```

17 #define MIN_DISTANCE 10
18 #define FOLLOW_DISTANCE 35
19
20 NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);
21
22 // Direction codes: F, B, L, R, S
23 void setMotors(char dir) {
24     int in1 = LOW, in2 = LOW, in3 = LOW, in4 = LOW;
25     switch (dir) {
26         case 'F': in1 = HIGH; in3 = HIGH; break;
27         case 'B': in2 = HIGH; in4 = HIGH; break;
28         case 'L': in1 = HIGH; in4 = HIGH; break;
29         case 'R': in2 = HIGH; in3 = HIGH; break;
30         default: break; // 'S' -> all LOW
31     }
32     digitalWrite(IN1, in1);
33     digitalWrite(IN2, in2);
34     digitalWrite(IN3, in3);
35     digitalWrite(IN4, in4);
36 }
37
38 void setup() {
39     pinMode(RIGHT_IR, INPUT);
40     pinMode(LEFT_IR, INPUT);
41     pinMode(ENA, OUTPUT);
42     pinMode(ENB, OUTPUT);
43     pinMode(IN1, OUTPUT);
44     pinMode(IN2, OUTPUT);
45     pinMode(IN3, OUTPUT);
46     pinMode(IN4, OUTPUT);
47     setMotors('S');
48 }
49
50 void loop() {
51     int distance = sonar.ping_cm();
52     if (distance == 0) distance = 200;
53
54     int rightIR = digitalRead(RIGHT_IR);
55     int leftIR = digitalRead(LEFT_IR);
56
57     // Dynamic speed control
58     int speedValue = (distance > 25) ? 220 : 170;
59     analogWrite(ENA, speedValue);
60     analogWrite(ENB, speedValue);
61
62     // Obstacle too close -> back off and turn
63     if (distance < MIN_DISTANCE) {
64         setMotors('S'); delay(300);
65         setMotors('B'); delay(400);
66         setMotors('S'); delay(200);
67         setMotors('R'); delay(300);
68         setMotors('S');
69         return;
70     }
71
72     if (rightIR == LOW && leftIR == HIGH) {
73         setMotors('R'); // turn right
74     } else if (rightIR == HIGH && leftIR == LOW) {
75         setMotors('L'); // turn left
76     } else if (distance >= MIN_DISTANCE && distance <= FOLLOW_DISTANCE) {
77         setMotors('F'); // follow object
78     } else {
79         setMotors('R'); // search mode
80     }
81 }

```

Listing 4.1: Arduino UNO Object Following Robot Control Code

Chapter 5 : Experimental Results, Conclusions and Future Scope

5.1 System Calibration and Testing

Testing began by calibrating the ultrasonic and infrared sensors. The ultrasonic sensor was tested against a flat target at distances from 5 cm to 100 cm to verify ranging accuracy. The infrared sensors were calibrated using their onboard potentiometers to trigger when an object was within 8 cm, preventing false detections from ambient light. Table 5.1 outlines the sensor calibration data and command mappings. Figure 5.1 shows the completed physical prototype, and Figure 5.2 shows the internal chassis wiring.

Ultrasonic (cm)	Left IR State	Right IR State	Motor Action
Distance < 10	High (No Refl)	High (No Refl)	Safety Recovery (Back & Turn Right)
$10 \leq \text{Dist} \leq 35$	High (No Refl)	High (No Refl)	Drive Forward (Fwd speed)
Distance ≥ 10	High (No Refl)	Low (Refl)	Pivot Right (Turn Speed)
Distance ≥ 10	Low (Refl)	High (No Refl)	Pivot Left (Turn Speed)
Distance > 35	High (No Refl)	High (No Refl)	Pivot Right (Search Mode)

Table 5.1: Sensor Calibration and Command Mapping

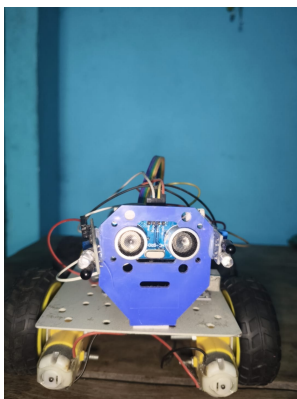


Figure 5.1: Assembled Object Following Robot

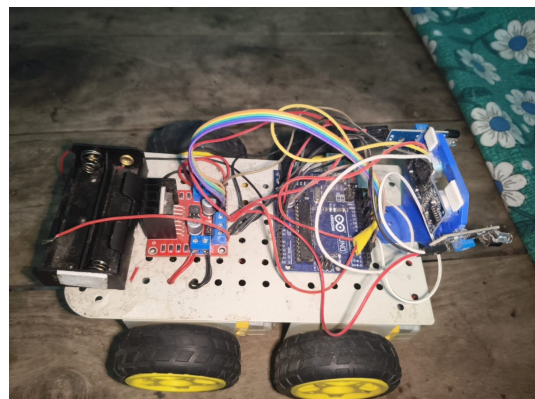


Figure 5.2: Internal Chassis Wiring and Layout

5.2 Performance Evaluation

The robot's tracking performance was evaluated in an indoor environment. Ranging trials showed that the ultrasonic sensor maintained an accuracy of ± 0.5 cm within the active tracking envelope (10 cm

to 35 cm). The control loop latency, including sensor readings, was measured at 10 ms, enabling the robot to respond to target movements in real time. The 7.4V battery pack supplied sufficient power for 50 minutes of active operation. Figures 5.3 and 5.4 show the testing setup.

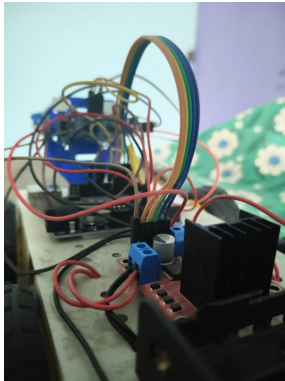


Figure 5.3: Ranging and Sensor Test Setup

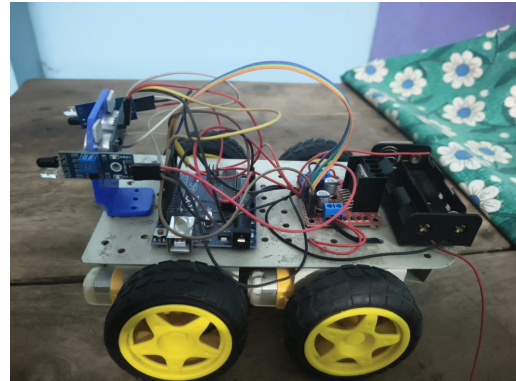


Figure 5.4: Battery Mount

5.3 Conclusions

This mini project successfully demonstrates the design, construction, and evaluation of an autonomous Object Following Robot using the Arduino UNO microcontroller. By combining an HC-SR04 ultrasonic sensor with dual infrared proximity sensors, the robot detects and follows forward and lateral target movements. The physical prototype achieved the design goals, maintaining a safety margin of 10 cm and steering to match target speeds. In summary, the multi-sensor design provides a low-cost, power-efficient alternative to vision-based tracking systems, showing that cooperative sensor control can achieve reliable performance on low-power hardware.

5.4 Future Scope

Although we removed the display and the servo motor from the final prototype due to power and voltage issues, future versions can add them back by improving the circuit:

1. Re-integrating the SG90 micro servo motor to rotate the ultrasonic sensor, allowing it to scan the environment up to 180 degrees.
2. Re-introducing the 0.96-inch SSD1306 OLED display to show the target distance and robot actions in real time.
3. Adding an LM2596 buck converter to isolate the servo's power draw from the microcontroller and prevent resets.
4. Upgrading the microcontroller to an ESP32 for wireless control and camera-based human tracking (ESP32-CAM).
5. Implementing a PID control loop to make the robot accelerate and decelerate more smoothly.

References

- [1] Arduino LLC, “Arduino UNO Hardware Reference Design,” Arduino Documentation, 2018. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3>.
- [2] InvenSense & Cytron Technologies, “HC-SR04 Ultrasonic Distance Sensor Product User Guide,” 2013. [Online]. Available: <https://www.electroschematics.com/wp-content/uploads/2013/07/HCSR04-Datasheet.pdf>.
- [3] S. Rajesh, V. Kumar, and M. Dev, “Multi-Sensor Design for Autonomous Human Following Mobile Robots,” *Int. J. Robot. Autom.*, vol. 12, no. 3, pp. 201–209, 2021. DOI: 10.11591/ijra.v12i3.2045.
- [4] STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet,” SGS-THOMSON Microelectronics, Jan. 2000. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>.
- [5] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Upper Saddle River, NJ: Pearson, 2005. ISBN: 0201543613.
- [6] P. Sinha and K. R. Sharma, “Sensor Fusion and Decoupling Control in Autonomous Wheeled Mobile Robots,” *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 5, pp. 1822–1830, 2020. DOI: 10.1109/TCST.2020.2983712.